

Trabajo Práctico Integrador — Etapa 2

Implementación y demostración · Comunicación de Datos 2025 · Comisión S33

00 Integrantes del grupo

Santoro, José Joaquín
Legajo 33647

Aramburu, Tomás
Legajo 34650

Pagotto, Ignacio
Legajo 33605

Leguizamón, Nicolás
Legajo 33592

01 Resumen de la entrega

A partir de la arquitectura y los mockups definidos en la Etapa 1, se desarrolló la aplicación completa: un front-end en React que consume una API REST construida con FastAPI. El back-end implementa el muestreo y la cuantización de imágenes (Pillow). Esta entrega incluye el código fuente completo, el instrumento de ejecución para que la cátedra pueda probar el desarrollo, y el enlace al video demostrativo.

02 Funcionalidades implementadas

Propuesta 3 — Digitalización de imágenes

- ✓ Carga de imágenes en alta resolución desde el navegador.
- ✓ Muestreo con distintos niveles de resolución (100×100, 500×500, 1000×1000), manteniendo la relación de aspecto.
- ✓ Reducción de profundidad de color a 1, 8 o 24 bits por canal.
- ✓ Comparación visual lado a lado: imagen original vs. digitalizada.
- ✓ Compresión del resultado (JPEG con calidad reducida) e indicador de tamaño de archivo (original vs. digitalizado, con % de reducción).

03 Código fuente

El código fuente completo se encuentra organizado en dos proyectos independientes dentro del repositorio, separando claramente front-end y back-end, y dentro de cada componente, separando JS/JSX de CSS:

```
backend/app/main.py · routers/imagen.py
backend/app/services/imagen_service.py
backend/app/models/schemas.py
frontend/src/App.jsx + App.css
frontend/src/components/ImagenDigitalizador/*.jsx + *.css
frontend/src/api/client.js
frontend/src/styles/index.css + shared.css
```

Repositorio público de GitHub: **[COMPLETAR CON EL LINK AL REPOSITORIO]**

04 Instrumento de ejecución

Opción 1 — Demo en vivo (sin instalación)

Aplicación desplegada para que la cátedra pueda probarla abriendo un link, sin instalar nada:

Frontend: **[COMPLETAR CON LA URL DE VERCEL]**
Backend (Swagger): **[COMPLETAR CON LA URL DE RENDER]/docs**

Nota: el backend está alojado en el plan free de Render; si lleva varios minutos sin uso puede demorar entre 30 y 50 segundos en responder la primera petición mientras el servicio se reactiva.

Opción 2 — Ejecución local

Como respaldo, el repositorio incluye un **README.md** con las instrucciones detalladas para correr el proyecto de forma local. En resumen:

Back-end (FastAPI):

```
cd backend
python -m venv venv
venv\Scripts\activate (Windows) / source venv/bin/activate (Linux/Mac)
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Front-end (React + Vite):

```
cd frontend
npm install
npm run dev
```

El front-end queda disponible en <http://localhost:5173> y el back-end (con documentación interactiva Swagger) en <http://localhost:8000/docs>. Requisitos: Python 3.10+ y Node.js 18+.

05 Evidencia de funcionamiento

Pruebas realizadas sobre la API durante el desarrollo: para la digitalización de imágenes se probó una imagen de 800×600 px (2.7 KB); muestreada a 100×100 con 8 bits por canal y compresión activada, el resultado ocupó 1.3 KB.

06 Video demostrativo

Enlace al video (máximo 6 minutos) que muestra el uso de las funcionalidades desarrolladas: **[COMPLETAR CON EL LINK AL VIDEO]**